PiXY: Pixel to stage-XY Coordinate Converter

Yoshiaki KON
Geological Survey of Japan (GSJ), National Institute of Advanced Industrial Science and Technology (AIST)
Central 7, 1-1-1, Higashi, Tsukuba, Ibaraki, Japan, 305-8567
Yoshiaki-kon@aist.go.jp

Abstract

PiXY is open-source software that links target selection on microscopy images with physical positioning on analytical-instrument stages in microanalysis workflows for geoscience and materials science, thereby reducing manual time and human error. PiXY automatically extracts particle centroids (u,v)(u,v)(u,v) by combining K-means color segmentation with connected-component analysis, and estimates a least-squares affine transform from user-acquired fiducials to convert image coordinates into instrument stage coordinates (X,Y,Z).

PiXY supports offline targeting, enabling users to select targets on pre-acquired images and prepare stage coordinates before instrument time. This shifts workload away from the instrument, shortens non-measurement stage operations, and improves instrument utilization. No dedicated fiducial markers or special sample preparation are required; repeatably identifiable sample features can be used as fiducials.

Validation on real specimens (N=100, five fiducials) evaluated the on-instrument reaching error (residual) for positions specified on images. Fifty percent of residuals were within 3 μm (X), 3 μm (Y), and 4 μm (Z), and 90% were within 10 μm (X), 16 μm (Y), and 21 μm (Z), demonstrating accuracy sufficient for practical operation.

Keywords
centroid extraction; fiducial-based transform; offline targeting; LA-ICP-MS; microscopy targeting; image registration;

Metadata

Nr	Code metadata description	***Metadata***
C1	Current code version	*1.3.0*
C2	Permanent link to code/repository used for this code version	*https://github.com/YoshiakiKON/PiXY*
C3	Permanent link to reproducible capsule	*https://doi.org/10.5281/zenodo.18174474*
C4	Legal code license	*MIT License*
C5	Code versioning system used	*git*
C6	Software code languages, tools and services used	*python*
C7	Compilation requirements, operating environments and dependencies	*https://github.com/YoshiakiKON/PiXY*
C8	If available, link to developer documentation/manual	*https://github.com/YoshiakiKON/PiXY/tree/main/documentation/InstructionManual_EN.md*
C9	Support email for questions	Yoshiaki-kon@aist.go.jp

1. Motivation and significance

Microanalysis instruments such as LA-ICP-MS, SIMS, EPMA, and SEM-EDS require micrometer-scale selection of analysis positions on solid sample surfaces. In a typical workflow, the surface is imaged in advance using optical or electron microscopy, and the same sample is then mounted on an analytical instrument. Operators adjust the XYZ stage while viewing the instrument's live image to determine measurement positions. In practice, this "positioning/targeting" step can take longer than the per-spot measurement itself, and the required time often depends on operator experience; consequently, targeting frequently becomes a major bottleneck of instrument time.

To address this issue, high-accuracy registration methods using dedicated fiducial markers have been proposed [1], but they may require pre-marking on the sample and can hinder adoption. Commercial tools also allow users to specify analysis positions on images and relocate them on the instrument; such tools sometimes use image-based fiducial points for coordinate transformation.

However, many existing tools still require manual specification of each measurement point, which is inefficient when targeting must handle numerous candidates obtained by automatic particle detection and centroid extraction. In addition, if a tool restricts the number of fiducials to only two or three, it becomes difficult to use four or more fiducials and constrain the transform by least squares to improve robustness and accuracy. Hence, an integrated GUI workflow that links established methods and executes the full process consistently remains lacking. While individual elements such as particle recognition and centroid extraction (image processing), and coordinate transformation based on fiducial points (least-squares estimation of an affine transform), are well established as algorithms, in practical targeting a unified GUI is needed that supports the complete sequence: generating many candidates on an image, iteratively updating the transform while entering fiducials and inspecting residuals, and finally exporting an instrument-ready coordinate list. PiXY integrates this sequence in a single application, enabling trial-and-error and coordinate preparation to be completed before instrument time.

In this work, we present PiXY and provide an open-source GUI workflow from particle detection (centroid extraction) through coordinate transformation and batch export of target coordinates. With PiXY, part of the targeting workload on the analytical instrument can be shifted to offline targeting (selecting targets and preparing coordinates before instrument time), thereby shortening non-measurement stage operations and improving measurement throughput.

2. Software description

2.1. Software architecture*:*

PiXY is a desktop GUI application that unifies, in a single workflow, (1) generating candidates (particle regions and centroids) from pre-acquired images and (2) converting image coordinates into analytical-instrument stage coordinates based on fiducial points, followed by export of instrument-ready coordinate tables (Figure 1). Internally, PiXY comprises GUI components (image display and overlay rendering; table editing for candidates and fiducials), image processing (K-means–based color segmentation [2], connected-component analysis, centroid calculation), transform

estimation (least-squares estimation from fiducial pairs with residual evaluation), and input/output (image loading; CSV/clipboard export; project save/load). To keep the interface responsive, computationally intensive steps are performed on a downscaled "processing image," and the resulting centroids are mapped back to full-resolution pixel coordinates for output. The project file format (.pixy) stores processing conditions and intermediate results, enabling exact reruns under identical settings for the same input image.

PiXY is implemented in Python 3.8+. The GUI uses PySide6 (Qt for Python) [3], and image processing relies on OpenCV [4] and NumPy [5].

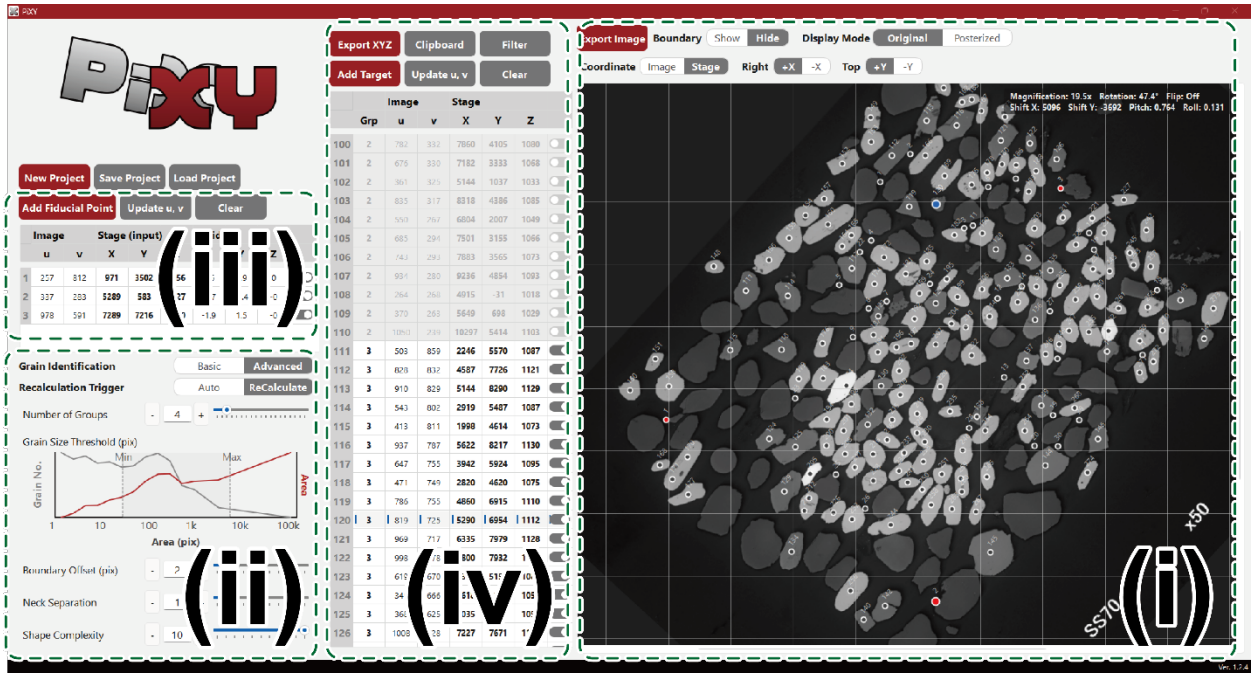

Figure 1: Example of the PiXY GUI showing (i) particle recognition and centroid overlays, (ii) particle-recognition and centroid-extraction parameters, (iii) fiducial points and residuals, and (iv) exportable coordinate tables.

2.2. Software functionalities:

PiXY supports a consistent workflow from offline candidate generation to instrument-ready coordinate list creation. Users load microscope or SEM images and extract particle regions via simple K-means–based color segmentation and connected-component analysis, obtaining centroid coordinates as candidates. Results are overlaid on the GUI as region boundaries and centroids, enabling interactive adjustment of key parameters and immediate inspection for over-segmentation, missed detections, and noise inclusion. Candidates are managed in a table, and exporting indexed overlay images and centroid lists facilitates record keeping.

Next, users mark repeatably identifiable sample features as fiducials on the image and enter the corresponding stage coordinates measured on the analytical instrument. PiXY estimates a 2D-to-3D affine approximation by least squares from multiple fiducial

pairs and visualizes per-fiducial residuals (errors), supporting outlier detection, re-identification, and re-estimation by adding or replacing fiducials. Using the estimated transform, PiXY converts candidate centroids into stage coordinates and exports them in convenient formats (CSV save, clipboard copy).
To assist fiducial re-identification, PiXY provides image rotation and flip operations. The analysis state can be saved and reloaded as a project file (.pixy), allowing integrated management of the image, processing settings, extraction results, fiducials, and transform outputs. The source code is publicly available on GitHub and is persistently archived on Zenodo (DOI: 10.5281/zenodo.18174474). PiXY can be run from Python by installing the dependencies listed in requirements.txt and launching Main.py; in addition, a standalone Windows executable built with PyInstaller is provided.

3. Illustrative examples

The following describes a typical workflow: users load a pre-acquired microscope image, estimate a coordinate transform from 3–5 fiducial points, and export instrument-ready stage coordinates
:

3.1. Offline targeting*:*

3.1.1 Image acquisition
Acquire an image of the sample surface using optical or electron microscopy.

3.1.2 Load the image
Launch PiXY on a Windows PC. Because a demo image is loaded at startup, select "New Project" and load a pre-acquired microscope image (e.g., a BSE image) into PiXY (Figure 2). After loading, particle recognition and centroid extraction run automatically, and particle regions and centroids are overlaid on the image.

3.1.3 Centroid extraction (particle recognition)
Tune parameters such as Number of Groups (K) and Grain Size Threshold (area thresholds) to extract particle regions and centroids (Figure 2). Because results are overlaid on the image, users can immediately check for over-segmentation, missed detections, and noise. If needed, Boundary Offset, Neck Separation, and Shape Complexity can suppress spurious regions and adjust splitting of touching particles.

3.1.4 Save the data
Select "Save Project" to save the input image, processing settings, and extracted centroids as a project file (.pixy, JSON format). In addition, "Export Image" saves an overlay image with centroid indices drawn on the original image.

3.2. Online targeting on the analytical instrument*:*

3.2.1 Load offline-targeting results
Launch PiXY on a Windows PC. If possible, run PiXY on the instrument-control PC to simplify transferring coordinates into the control application. Select "Load Project" and open the project saved in 3.1.4. If working on the same PC, continue the workflow as is.

3.2.2 Enter fiducial points
Switch to fiducial-entry mode ("Add Fiducial Point") and click repeatably identifiable features on the image (e.g., particle tips, scratches, edges) to add fiducials (Figure 2).

On the analytical instrument, relocate the same features, measure stage coordinates (X, Y, Z), and enter them into the fiducial table. PiXY also provides image rotation/flip to assist fiducial identification.

3.2.3 Inspect residuals and re-estimate

After entry, PiXY estimates the coordinate transform by least squares from multiple fiducial pairs and displays per-fiducial residuals (errors). If a fiducial shows a large residual, re-identify it, add or replace fiducials, or exclude the outlier and re-estimate.

3.2.4 Export coordinate data

Export the converted target information—spot index, group index, and stage coordinates (X, Y, Z)—in instrument-friendly formats (CSV save, clipboard copy). Load or paste the data into the instrument's coordinate input file to import coordinates.

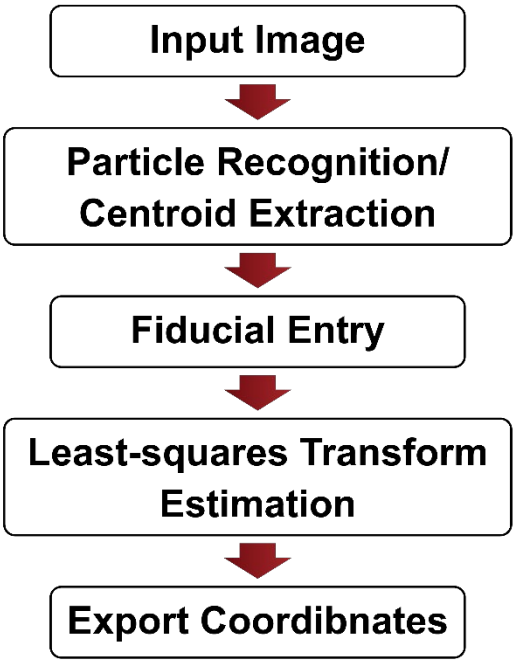

Figure 2: Workflow in PiXY (image loading → centroid extraction → fiducial entry → residual inspection → coordinate export).

3.3. Validation of coordinate transformation:

We validated PiXY's coordinate transformation using real specimens and microanalysis instruments. BSE images were acquired with a scanning electron microscope (JEOL JSM-6610LV) and a laser-ablation system (Raijin; Seishin Shoji). A representative dataset was obtained at 15× magnification (1560 × 1920 pixels; 3.33 μm/pixel) from a polished epoxy mount. For fiducials, stage coordinates were recorded on the instrument while repeatedly relocating the same features; XY positions were read when the feature was centered, and Z was recorded after focus adjustment (autofocus or equivalent).

For evaluation, the BSE image was loaded into PiXY and centroids were extracted using standard parameters (K = 5; minimum area 20 px; maximum area 4000 px). Fiducials were chosen near the mount rim to sufficiently constrain the transform; two configurations were compared: three fiducials (approximately 120° spacing) and five fiducials (approximately 70° spacing). After estimating the transform, stage coordinates exported by PiXY were sent to the instrument to move the stage. The difference between intended and reached positions was evaluated as residuals. Measurement points spanned approximately 5000 µm in X and Y and 100–400 µm in Z.

Each configuration was evaluated in three runs with different sample orientations (rotation/tilt). In each run, 30–40 target points were measured, yielding N = 100 residuals per configuration (Figure 3). With five fiducials, 50% of residuals were within 3 µm (X), 3 µm (Y), and 4 µm (Z), and 90% were within 10 µm (X), 16 µm (Y), and 21 µm (Z). With three fiducials, 50% were within 11 µm, 12 µm, and 6 µm, and 90% were within 32 µm, 42 µm, and 20 µm, for X, Y, and Z, respectively. In general, increasing the number of fiducials strengthens the least-squares constraint and tends to reduce both bias and variability.

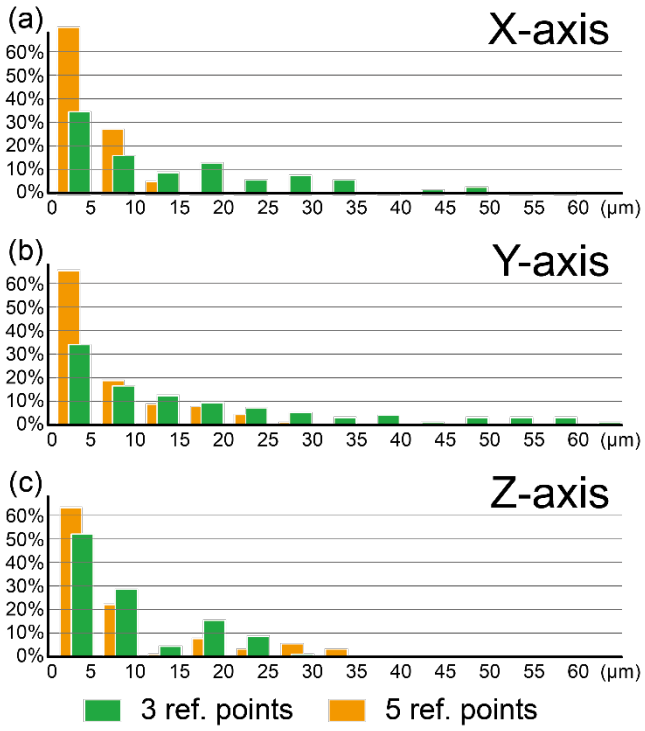

Figure 3: Residual histograms for each axis (X, Y, Z) comparing three vs. five fiducial points.

4. Impact

The primary impact of PiXY is shifting the targeting step in microanalysis from on-instrument manual work to offline preparation, thereby reducing the burden of positioning and re-identification that often limits machine time. On analytical instruments with limited fields of view, exploring and positioning micrometer-scale targets can be time-consuming, operator-dependent, and error-prone, particularly when many targets must be located. PiXY integrates candidate generation (centroid extraction) on pre-acquired images with fiducial-based

coordinate transformation, residual inspection, and coordinate export in a single GUI workflow, supporting a reduction of non-measurement stage operations.

(1) New research questions enabled: By making it straightforward to prepare large numbers of measurement candidates before instrument loading, PiXY supports a shift from “a small number of representative spots” to “statistical evaluations over many spots.” Examples include dense sampling of particle populations and phase boundaries, sampling strategies informed by particle size/shape/composition correlations, and systematic evaluation of how fiducial placement and the number of fiducials affect residuals (optimization of fiducial operations). Compared with automatic registration approaches that rely on dedicated markers (e.g., autoCRIM [1]), PiXY assumes operation without dedicated markers and focuses on workflow optimization under practical constraints.

(2) Improvements over existing practice: In many microanalysis workflows, relocation is a bottleneck that constrains the number of measured points and reduces instrument efficiency. PiXY (i) automatically generates candidates on the image, (ii) allows the use of four or more fiducials to better constrain least-squares estimation, and (iii) supports iterative re-identification and re-estimation while inspecting residuals—concentrating trial-and-error into offline work. In validation (Section 3.3), with five fiducials, 50% of residuals were within 3 μm (X), 3 μm (Y), and 4 μm (Z), and 90% were within 10 μm (X), 16 μm (Y), and 21 μm (Z), indicating practical accuracy and showing that increasing fiducials improves residual distributions.

As an operational example, configuring approximately 200 measurement points can take more than one hour with conventional manual relocation on the instrument, whereas using PiXY with offline targeting reduced preparation time to ~20 minutes in a practical case, lowering the barrier to higher-throughput measurements (e.g., >10^3 points).

(3) Changes in day-to-day practice: PiXY manages candidates and fiducials as tables and lets users correct misidentifications or outliers while inspecting overlays and residuals. Because the analysis state can be saved/loaded as a project file (.pixy), hand-offs among personnel and later re-analysis/re-export (e.g., adding targets or fiducials and re-estimating) are simplified. Fixing the random seed for K-means and saving processing settings further support the reproducibility of procedures and outputs for the same input image.

(4) Adoption and dissemination: PiXY is publicly available on GitHub and persistently archived on Zenodo (DOI: 10.5281/zenodo.18174474). As the public release is recent, quantitative metrics such as citation counts are not yet informative; however, software can be cited by DOI with version specifications to ensure reproducible use. PiXY is distributed both as Python source and as a standalone Windows executable, enabling use on instrument-control PCs without a Python environment. Download and usage statistics can be tracked via GitHub/Zenodo for future adoption assessments.

(5) Commercial use and spin-offs: Because PiXY does not depend on manufacturer-specific control APIs or dedicated markers, it can be used in commercial environments (contract analysis, quality control, research services) as a tool to streamline targeting that consumes instrument time. At present, no spin-off companies or productizations based on PiXY are known.

Finally, because particle-recognition accuracy for centroid extraction depends on image simplicity and contrast, PiXY accepts externally preprocessed images (e.g., segmentation or

binarization). Future work includes integrating segmentation modules and further automation through instrument-control APIs.

5. Conclusions

PiXY integrates candidate generation (centroid extraction) and fiducial-based coordinate transformation into a single GUI workflow for practical offline targeting in microanalysis, thereby reducing manual relocation during instrument time.

Validation with real specimens shows that increasing the number of fiducial points improves residual distributions, confirming that PiXY achieves accuracy sufficient for practical operation. Future work includes integrating preprocessing/segmentation and further automation via instrument-control APIs to reduce targeting effort.

Acknowledgements

This work was supported by JSPS KAKENHI (Grant Number: 25H00682). The author acknowledges the open-source communities behind Python, OpenCV, NumPy, and PySide6.

During development, the author used AI-assisted tools (GitHub Copilot, Google Gemini, xAI Grok, Anthropic Claude) for manuscript editing suggestions and programming support. All generated outputs were reviewed and validated by the author, who takes full responsibility for the final content.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used GitHub Copilot, Google Gemini, xAI Grok, and Anthropic Claude in order to assist with drafting and programming tasks. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] Sheriff J, Fletcher IW, Cumpson PJ. Computer-readable image markers for automated registration in correlative microscopy (autoCRIM) [preprint]. arXiv:2011.14949. 2020. https://doi.org/10.48550/arXiv.2011.14949.

[2] MacQueen JB. Some methods for classification and analysis of multivariate observations. In: Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics. Berkeley, CA: University of California Press; 1967, p. 281-97.

[3] The Qt Company. Qt for Python (PySide6) [software]. 2024. https://wiki.qt.io/Qt_for_Python; [accessed 29 January 2026].

[4] Bradski G. The OpenCV library. Dr Dobb's Journal of Software Tools. 2000.

[5] Harris CR, Millman KJ, van der Walt SJ, Gommers R, Virtanen P, Cournapeau D, et al. Array programming with NumPy. Nature. 2020;585(7825):357-62. https://doi.org/10.1038/s41586-020-2649-2.